Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Practica:

Sincronizacion

Profesora:

M.I. Elba Karen Saenz Garcia

Fecha de Entrega:

17 de Abril del 2026

Índice

1 Desarrollo: **1**
1.1 Algoritmo de Cristian: 1
1.1.1 Actividad 1 1
1.1.2 Actividad 2 1
1.2 Algoritmo de Berkeley: 2
1.2.1 Notas: 2
1.2.2 Actividad 3 3
1.2.3 Actividad 4 5
1.2.4 Actividad 5 7
1.3 Algoritmo de Lamport: 8
1.3.1 Actividad 6 8
1.3.2 Actividad 7 8

Referencias **8**

1. Desarrollo:

1.1. Algoritmo de Cristian:

1.1.1. Actividad 1

Simular latencia en red agregando

```
delay = 2 # segundos
time.sleep(delay)
```

¿Como cambia el ajuste del reloj?

1.1.2. Actividad 2

- **Modificar la implementación anterior para que se pueda trabajar con varios clientes al mismo tiempo (máquinas diferentes, pueden ser físicas o virtuales y multithread) y se vea una diferencia de tiempo.**
- **¿Todos obtienen el mismo ajuste? ¿Por qué?**
- **Agregar al código lo necesario para considerar cuando C > Cc o C<Cc.**

1.2. Algoritmo de Berkeley:

1.2.1. Notas:

- Cada cliente inicia con un reloj distinto
- El servidor calcula un promedio
- Todos convergen a un mismo tiempo
- Ejecutar el programa y responder:
    1. **¿Cuál era el tiempo inicial de cada cliente?**
    2. **¿Cuál fue el promedio?**
    3. **¿A qué tiempo convergen?**

Salida del programa:

```
Servidor de Berkeley escuchando en el puerto 60001...
Conexion establecida con: ('127.0.0.1', 47948)
Conexion establecida con: ('127.0.0.1', 47954)
Conexion establecida con: ('127.0.0.1', 47968)
Tiempo recibido del cliente: 1774828614.6528187
Tiempo recibido del cliente: 1774828611.847464
Tiempo recibido del cliente: 1774828616.3387685
Tiempo del servidor: 1774828613.939467
Tiempo promedio calculado por el servidor es: 1774828614.1946297
Ajuste del servidor: [-0.458, 2.347, -2.144]

Cliente 1 - Tiempo local antes del ajuste: 1774828611.847464
Cliente 1 - Ajuste recibido: 2.347165584564209
Cliente 1 - Nuevo tiempo despues del ajuste: 1774828614.1946297

Cliente 2 - Tiempo local antes del ajuste: 1774828614.6528187
Cliente 2 - Ajuste recibido: -0.4581890106201172
Cliente 2 - Nuevo tiempo despues del ajuste: 1774828614.1946297

Cliente 3 - Tiempo local antes del ajuste: 1774828616.3387685
Cliente 3 - Ajuste recibido: -2.144138813018799
Cliente 3 - Nuevo tiempo despues del ajuste: 1774828614.1946297
```

Respuestas:

1. **¿Cuál era el tiempo inicial de cada cliente?**

Los tiempos iniciales de cada cliente fueron:

- Cliente 1: 1774828611.847464 (2.35 segundos atrás del promedio)
- Cliente 2: 1774828614.652818 (0.46 segundos adelante del promedio)
- Cliente 3: 1774828616.338768 (2.14 segundos adelante del promedio)

Cada cliente simula una variación aleatoria de ±5 segundos respecto al tiempo real del sistema.

2. **¿Cuál fue el promedio?**

   El tiempo promedio calculado por el servidor fue **1774828614.1946297**. Este valor se obtiene promediando los tiempos de los 3 clientes más el tiempo del servidor (1774828613.939467), resultando en: $(611.84 + 614.65 + 616.34 + 613.94)/4 = 614.19$ (últimos dígitos significativos).

3. **¿A qué tiempo convergen?**

   Todos los clientes convergen al tiempo promedio **1774828614.1946297**. Como se puede observar en la salida, después de aplicar los ajustes correspondientes, el "Nuevo tiempo después del ajuste" es idéntico para los tres clientes, demostrando que el algoritmo de Berkeley logra sincronizar exitosamente todos los relojes al mismo valor.

1.2.2. Actividad 3

Para esta actividad se modificó el programa original para soportar un número variable de clientes. Los cambios principales fueron: (1) parametrizar la función `servidorBerkeley` y `ejecutaCliente` para recibir el número de clientes como argumento, (2) agregar la opción `SO_REUSEADDR` al socket para evitar errores de "address already in use" entre ejecuciones, (3) usar puertos diferentes para cada prueba (60001, 60002, 60003) evitando conflictos, y (4) crear una función `ejecutarPrueba` que encapsula la ejecución completa de servidor y clientes para cada configuración.

Modificar el programa para trabajar con:

- 5, 10 y 20 clientes
- Respondera las preguntas:
  1. **¿Cómo cambia el tiempo promedio?**
  2. **¿El sistema sigue siendo estable?**

Salida del programa con 10 clientes:

```
============================================================
```

```
                      SERVIDOR BERKELEY
============================================================
| Numero de clientes esperados: 10
| Puerto: 60002
============================================================

[SERVIDOR] Tiempos recibidos de clientes:
[SERVIDOR]   Cliente 01: 1774828674.637297
[SERVIDOR]   Cliente 02: 1774828676.781445
[SERVIDOR]   Cliente 03: 1774828670.725160
[SERVIDOR]   Cliente 04: 1774828674.653284
[SERVIDOR]   Cliente 05: 1774828678.582883
[SERVIDOR]   Cliente 06: 1774828675.318715
[SERVIDOR]   Cliente 07: 1774828670.358184
[SERVIDOR]   Cliente 08: 1774828678.589895
[SERVIDOR]   Cliente 09: 1774828678.777744
[SERVIDOR]   Cliente 10: 1774828680.287497
[SERVIDOR]   Servidor:   1774828675.524798

[SERVIDOR] Tiempo promedio: 1774828675.839719

[SERVIDOR] AJUSTES CALCULADOS
[SERVIDOR]   Cliente 01: +1.202421 segundos
[SERVIDOR]   Cliente 02: -0.941726 segundos
[SERVIDOR]   Cliente 03: +5.114559 segundos
[SERVIDOR]   Cliente 04: +1.186435 segundos
[SERVIDOR]   Cliente 05: -2.743164 segundos
[SERVIDOR]   Cliente 06: +0.521003 segundos
[SERVIDOR]   Cliente 07: +5.481535 segundos
[SERVIDOR]   Cliente 08: -2.750176 segundos
[SERVIDOR]   Cliente 09: -2.938025 segundos
[SERVIDOR]   Cliente 10: -4.447779 segundos
[SERVIDOR]   Servidor:   +0.314920 segundos

[SERVIDOR] SINCRONIZACION COMPLETADA
```

Respuestas:

1. **¿Cómo cambia el tiempo promedio?**

   El tiempo promedio se vuelve más representativo del estado global del sistema conforme aumenta el número de clientes. Con 3 clientes, el promedio se ve más influenciado por valores extremos (outliers), mientras que con 10 o 20 clientes, el efecto de un reloj muy adelantado o atrasado se diluye. En las pruebas realizadas, el promedio tiende a acercarse más al tiempo real del servidor cuando hay más

participantes, ya que las variaciones aleatorias (±5 segundos) tienden a cancelarse estadísticamente.

2. **¿El sistema sigue siendo estable?**

   Sí, el sistema permanece estable con 5, 10 y 20 clientes. El algoritmo de Berkeley es inherentemente escalable porque: (1) el cálculo del promedio es una operación O(n) simple, (2) cada cliente solo requiere una conexión TCP al servidor, y (3) el intercambio de mensajes es mínimo (solicitud de tiempo, envío de tiempo, recepción de ajuste). En todas las pruebas, todos los clientes convergieron exitosamente al tiempo promedio calculado. La única consideración es el tiempo de espera del servidor para recibir todos los tiempos, que aumenta proporcionalmente con el número de clientes.

1.2.3. Actividad 4

Para simular la latencia de red, se agregó un retraso aleatorio en la función `ClienteBerkeley` usando `time.sleep(random.uniform(0, 3))`. Este retraso se aplica en dos momentos: (1) antes de enviar el tiempo local al servidor (simulando latencia de envío), y (2) antes de recibir el ajuste del servidor (simulando latencia de recepción). De esta manera, cada cliente puede experimentar una latencia total de 0 a 6 segundos, representando condiciones variables de red.

- Agrega un retraso aleatorio en los clientes:
  - time.sleep(random.uniform(0, 3))
- Responder a las preguntas:
  1. **¿La latencia afecta el resultado final?**
  2. **¿Por qué el algoritmo de Berkeley es sensible o no a este factor?**

Salida del programa con latencia simulada:

```
============================================================
ACTIVIDAD 4 - SIMULACION DE LATENCIA DE RED
============================================================
[CLIENTE 01] Tiempo local: 1774828782.780560 (var: -0.93s)
[CLIENTE 01] Latencia simulada: 2.674 seg
[CLIENTE 01] Ajuste: -0.138924 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 02] Tiempo local: 1774828779.546950 (var: -4.37s)
[CLIENTE 02] Latencia simulada: 1.777 seg
[CLIENTE 02] Ajuste: +3.094686 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 03] Tiempo local: 1774828783.538568 (var: -0.57s)
[CLIENTE 03] Latencia simulada: 0.232 seg
[CLIENTE 03] Ajuste: -0.896932 -> Nuevo tiempo: 1774828782.641636
```

```
[CLIENTE 04] Tiempo local: 1774828779.330222 (var: -4.99s)
[CLIENTE 04] Latencia simulada: 2.648 seg
[CLIENTE 04] Ajuste: +3.311414 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 05] Tiempo local: 1774828783.458510 (var: -1.06s)
[CLIENTE 05] Latencia simulada: 2.423 seg
[CLIENTE 05] Ajuste: -0.816874 -> Nuevo tiempo: 1774828782.641636

[SERVIDOR] Tiempo promedio: 1774828782.641636
[SERVIDOR] SINCRONIZACION COMPLETADA
============================================================
```

Respuestas:

1. **¿La latencia afecta el resultado final?**

   No, la latencia **no afecta el resultado final** de la sincronización en términos de convergencia. Como se puede observar en la salida, a pesar de que cada cliente experimentó diferentes latencias (desde 0.232 hasta 2.674 segundos), todos convergieron al mismo tiempo promedio (1774828782.641636). Esto se debe a que el algoritmo de Berkeley trabaja con los *valores de tiempo reportados* por cada cliente, no con el momento exacto en que esos valores llegan al servidor.

   Sin embargo, la latencia sí afecta la **precisión** del resultado. Si un cliente tiene alta latencia, el tiempo que reporta ya estará "desactualizado" cuando llegue al servidor, introduciendo un error sistemático. En nuestra implementación, el cliente captura su tiempo local *antes* de la latencia, por lo que el ajuste calculado es correcto para ese instante, pero cuando el cliente finalmente aplica el ajuste, su reloj real ya avanzó.

2. **¿Por qué el algoritmo de Berkeley es sensible o no a este factor?**

   El algoritmo de Berkeley es **parcialmente sensible** a la latencia por las siguientes razones:

   Insensibilidad: El algoritmo no depende de una fuente de tiempo externa (como NTP), sino que calcula un promedio democrático de todos los participantes. La latencia no impide que el algoritmo calcule y distribuya los ajustes correctamente.

   Sensibilidad: En redes con alta latencia o latencia asimétrica, el algoritmo puede introducir errores porque:

   - El tiempo que tarda un mensaje en llegar no se compensa (a diferencia del algoritmo de Cristian que usa RTT/2).

- Si la latencia varía significativamente entre clientes, algunos recibirán sus ajustes más tarde, acumulando desfase adicional.

Para mitigar esto, implementaciones robustas de Berkeley incorporan estimación de RTT (Round-Trip Time) y compensan la latencia al calcular los ajustes.

1.2.4. Actividad 5

Modificar la implementación anterior para ejecutar, servidor y clientes en máquinas diferentes.

1.3. Algoritmo de Lamport:

Para la implementación del algoritmo de relojes lógicos de Lamport se proporciona un programa en Python que utiliza el módulo multiprocessing para la generación de procesos locales.

1.3.1. Actividad 6

Nota: en las secciones que se pide ambiente propuesto, se tiene que cambiar a un ambiente que propongamos nosotros, la profa quiere que todos los ambies sean diferentes, por eso no podemos trabajar con el que ella quiere

- El programa es de acuerdo a un ambiente de eventos establecido de cada proceso.
- Se pide probarlo y entenderlo para poder plantear en equipo un ambiente sugerido con 7 procesos.
- Entregar diagrama de ambiente propuesto

1.3.2. Actividad 7

Nota: en las secciones que se pide ambiente propuesto, se tiene que cambiar a un ambiente que propongamos nosotros, la profa quiere que todos los ambies sean diferentes, por eso no podemos trabajar con el que ella quiere

Con la idea anterior implementar una solución al algoritmo de relojes vectoriales, plantear un ambiente con 5 procesos. Entregar la figura con el ambiente sugerido que se programará.

Referencias

Oracle. (s.f.). *Oracle VM VirtualBox - Downloads | Oracle Technology Network | Oracle*. https://www.oracle.com/virtualization/technologies/vm/downloads/virtualbox-downloads.html

Petrosez. (2025, 12 de septiembre). *PelicanHPC*. https://sourceforge.net/projects/pelicanhpc/

Saenz García, E. K. (2026, 3 de marzo). *ClusterHPC*. YouTube. https://www.youtube.com/watch?v=F_LFf0lhbRA